

1.

(a) Let's imagine you're working for Apple. With their fitness tracking features included with their Apple Watch, they want you to design an algorithm that will return a peak heart rate from a workout that was tracked with their fitness tracking. You will be given an array each minute and the heart rate at each minute. A peak is defined as an element of the array where the element to left, and right is less than that element. At the edges of the array consider if the first element is greater than the second, then it is a peak. If the last element is greater than the second to last, then it is a peak. If the array is of length 2, then it is safe to say the larger element is the peak. If a plateau occurs, it is not considered a peak, and you can decide how you want to search for the rest of the array. Of course, if the length of the array is only one, then that can be considered the peak. You are asked to find only one peak.

(b) I would expect a candidate to implement a binary search algorithm. As I am asking to only return one peak, there is no need to search for the entire array. By implementing binary search algorithms if the middle element is not a peak, then they can continue from that point and search the side that the element next to is greater than the element. They can continue this search until a solution is found. I believe that it can be solved this way because you are guaranteed to find a solution, even if it is the first or last element of the array.

(c)

1. Implementing binary search. The whole reason why I am asking to return only one peak is because I want to see that they can implement and think with efficient searching rather than just using a brute force method such as linear search. I would expect to implement using recursion or a loop. That is the easiest way that this can be solved for me.

2. Time complexity. Being able to analyze the time complexity of this algorithm correctly. I would expect the candidate to be able to explain why this is $O(\log(n))$. If I was giving an interview, this would just be a verbal conversation between the candidate and I, I would expect them to mention something about dividing the array as to why this belongs to $O(\log(n))$ family.

3. Correctness. Is the algorithm going to work in the majority if not all scenarios. I would expect the solution to work in all cases where inputs are. I would also expect the client to be able to explain why their solution is correct. I would expect them to be able to clearly explain why their solution is correct otherwise I may not believe that the solution is correct.

(d) If the candidate cannot explain the code/solution as to why it is correct. If the candidate cannot explain what their code/solution is doing, then I also may not be able to understand why they are correct. If they are also not able to explain why their solution is

correct, then I would think that even if it works for certain test cases it may not work for all.

I would also consider them not being able to solve this in $O(\log(n))$ to be a deal breaker. Being able to see that this is a Binary Search problem is something that I would need to see if I was an interviewer.

(e) Imagine a couple months later down the road, apple wants to change this algorithm to return the highest peak to the user. I would ask this question because there are multiple ways that this algorithm can be solved such as using a sorting algorithm or linear search for the maximum. I think it would give good insight into a candidate's thought process when solving this second algorithm.

(f)

- Incorrect Base cases. If a candidate does not handle arrays of smaller sizes like 1 and 2 correctly, this is significant because then this can lead to incorrect solutions possibly because of incorrect indexes or missing peaks that are valid, this can lead to evaluation of lack of detail when evaluating a client.
- Not considering all cases. If a candidate cannot implement all possible scenarios like the edge cases, and even all elements equal (plateau), then this can lead to incorrect implementation. This is similar to the base cases, in that some wrong implementations can lead to incorrect solutions, also shows lack of detail.
- Overcomplicating the solution. This can return unnecessary peaks or cause an error in time complexity. As this solution I am only looking for one peak so not just searching for one can lead me to think that a candidate is not one to listen to directions.
- Improper termination. Setting the recursion statements incorrectly can lead to possibly incorrect answers if a valid peak has been skipped. This is often a simple logic error; this is not as significant as the other pitfalls in my opinion, but can still impact a candidate's evaluation because although it a logic error is a simple fix, this means that they may not have taken the time to debug and find this error.

2.

Definition of Big O: Given two function f and g , if there are positive constant c and k such that $0 \leq f \leq cg$ for all $n \geq k$

1. let $f = 5n^2 - 15n + 548$ and $g = n^2$, $0 \leq 5n^2 - 15n + 548 \leq 553 * n^2$ for all $n \geq 1$, therefore $5n^2 - 15n + 548$ is $O(n^2)$

2. let $f = n!$ and $g = n^n$, $0 \leq n! \leq n^n$, for all $n \geq 1$,

Therefore $n!$ Belongs to $O(n^n)$

3.

```

# with number^n n is either even or odd
# an even number is simply n=2*k
# an odd number is simply n=2*k+1
# with this problem a the number 2^n is the same as
# (2^(n/2))^2 for even numbers and
# 2*(2^((n-1)/2))^2 for odd numbers
def function_name1(n):
    #base case
    if n==0:
        return 1
    elif n==1:
        return 2025

    half= function_name1(n//2)

    if n % 2 ==0:
        return half*half
    else:
        return 2025*half*half

```

- The way I implemented this algorithm was to know that an even number is $n=2k$, an odd number as $n=2k+1$. Instead of decrementing from n , leading $O(n)$, I was able to use the divide step knowing with the law exponents that I was able 2025^n if n was even would be equal to $(2025^{(n/2)})^2$, and if n was odd then $2025*(2025^{(n/2)})^2$ by law of exponents. This allowed me to have a divide, combine, and conquer.
- When I implemented this algorithm, my approach was to implement with 2^n because I know binary numbers well. With induction I was able to get the correct answer when I tested at $2^0, 2^1, 2^2, 2^3, 2^4, \dots, 2^8$ I got the correct answer so the answer correct for any 2^n . Once I changed the code to be for 2025^n using the same logic. The base case of $n=0$ returns which return 1 which is correct. The induction step of $n=k$, if $n=k+1$ either returns the number double for even numbers and multiplied by itself for odd numbers so correctness holds for all n .
- Time Complexity will be $O(\log(n))$ because I am skipping elements so it will not be $O(n)$. Since I am doing a constant amount of time by doing some multiplications

outside of recursion and I am making one recursion call dividing by 2 means $\log_2 n$ leading to $O(\log(n))$ time complexity.

4. I would implement a Min Heap. I would then allow this heap to only have the k elements that we want. If the heap is not full then I would select the first k elements and have to min-heapify after each one. As we iterate through the rest of the elements, we compare each element with the top of the heap. If an element is greater than the top element, then we would remove the top element and then select the element that was greater in that spot. Then call the min-heapify function to ensure that the min heap is maintained. I would then continue this process until we have compared each element and then return the ending k elements in the heap.